

HASHICORP CERTIFIED | EXAM CODE: 003

Terraform

Associate

003

200 Interview Questions & Answers

Complete exam preparation covering all exam domains:

IaC Concepts | State Management | Modules | Backends

Workspaces | Terraform Cloud | CLI Commands | Best Practices

200

Questions

16

Sections

003

Exam Code

~70%

Pass Score

Table of Contents

HashiCorp Terraform Associate 003 – Complete Interview Preparation Guide

#	Section	Key Topics	Q's
1	Terraform Basics & Core Concepts	IaC, HCL, providers, resources, data sources	21
2	Terraform State Management	State file, remote backend, locking, drift, import	17
3	Terraform Configuration Language	Variables, locals, outputs, count, for_each, dynamic	20
4	Plan, Apply & Lifecycle	Plan, apply, destroy, lifecycle, create_before_destroy	17
5	Terraform Modules	Modules, registry, inputs/outputs, versioning, reuse	15
6	Backends & Remote State	S3 backend, remote state, migration, encryption	8
7	Terraform Workspaces	Workspaces, OSS vs Cloud, environment isolation	5
8	Built-in Functions	lookup, merge, flatten, cidrsubnet, templatefile, toset	10
9	Security & Best Practices	Secrets, sensitive vars, lock file, provider pinning	8
10	Terraform Cloud & Enterprise	HCP Terraform, Sentinel, cost estimation, agents	10
11	Advanced Concepts	DAG, depends_on, moved blocks, check, CDKTF, test	10
12	Terraform CLI Commands	init, plan, apply, destroy, output, force-unlock	8
13	Exam Preparation (Terraform 003)	Exam format, domains, file types, OSS vs Enterprise	10
14	Provisioners & Connections	Provisioners, local-exec, remote-exec, file, connection	10
15	Terraform Registry & Providers	Registry, Official/Partner providers, aliases	10
16	Scenario-Based Questions	Corrupted state, CI/CD, multi-account, drift, rollback	21

200	16	003	~70%
Total Questions	Sections	Exam Code	Passing Score

1 Terraform Basics & Core Concepts

21 Qs

Q1 QUESTION

What is HashiCorp Terraform?

ANSWER

Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp that lets you define, provision, and manage infrastructure using HCL (HashiCorp Configuration Language). It supports multiple cloud providers through a plugin-based provider system, enabling consistent, repeatable infrastructure deployments.

Q2 QUESTION

What is Infrastructure as Code (IaC)?

ANSWER

IaC is the practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes. Benefits include version control, consistency, repeatability, automation, collaboration, and dramatic reduction in human error across environments.

Q3 QUESTION

What is HCL (HashiCorp Configuration Language)?

ANSWER

HCL is Terraform's native configuration language — human-readable yet machine-friendly. It supports expressions, functions, references, and conditionals. HCL files use the .tf extension and are used to declare the desired state of infrastructure.

Q4 QUESTION

What are the key benefits of Terraform?

ANSWER

Key benefits: (1) Multi-cloud/provider support, (2) Declarative syntax — describe what not how, (3) State management for drift detection, (4) Plan before apply — safe previews, (5) Dependency graph for parallel execution, (6) Modular and reusable code, (7) Huge provider/module ecosystem.

Q5 QUESTION

What is the Terraform core workflow?

ANSWER

The three-step workflow: Write ▢ define infra in .tf files; Plan ▢ 'terraform plan' previews changes safely; Apply ▢ 'terraform apply' executes changes. A fourth step 'terraform destroy' removes all managed resources.

Q6 QUESTION

What is a Terraform provider?

ANSWER

A provider is a plugin enabling Terraform to interact with cloud platforms, SaaS tools, and APIs. Each provider exposes resource types and data sources. Examples: aws, azure, google, kubernetes, github, datadog. Providers are versioned independently of Terraform CLI.

Q7 QUESTION

How do you initialize a Terraform project?

ANSWER

Run 'terraform init' in the project directory. It downloads required provider plugins, sets up backend configuration, downloads modules, and prepares the working directory. Must be run before any other Terraform commands and re-run after configuration changes.

Q8 QUESTION

What is a Terraform resource?

ANSWER

A resource is the fundamental building block representing a real infrastructure object (VM, S3 bucket, DNS record). Declared as: `resource "aws_instance" "web" { ami = "ami-123" instance_type = "t3.micro" }`. Each resource has a type and a local name.

Q9 QUESTION

What is a Terraform data source?

ANSWER

A data source reads information from existing infrastructure or external APIs without managing it: `data "aws_ami" "latest" { most_recent = true }`. The result is referenced as `data.aws_ami.latest.id`. Data sources are read-only — they never create or modify resources.

Q10 QUESTION

What is the difference between Terraform and Ansible?

ANSWER

Terraform provisions infrastructure declaratively (create VMs, networks, DBs). Ansible configures existing systems procedurally (install packages, edit configs). Terraform manages state; Ansible is stateless. Both complement each other — Terraform to provision, Ansible to configure.

Q11 QUESTION

What are Terraform meta-arguments?

ANSWER

Meta-arguments are special arguments supported by all resource types: `depends_on` (explicit dependencies), `count` (create N copies), `for_each` (create from map/set), `provider` (select non-default provider), `lifecycle` (`create_before_destroy`, `prevent_destroy`, `ignore_changes`, `replace_triggered_by`).

Q12 QUESTION

What is 'terraform fmt'?

ANSWER

'terraform fmt' formats all .tf files to canonical HCL style — consistent indentation, spacing, and alignment. Use `-check` in CI/CD to fail if formatting is incorrect. Use `-recursive` to format all subdirectories. Run before every commit.

Q13 QUESTION

What is 'terraform validate'?

ANSWER

'terraform validate' checks configuration for syntax errors and internal consistency without accessing remote services or state. It validates resource arguments, types, and references. Run after `fmt` as the first gate in a CI/CD pipeline.

Q14 QUESTION

What is 'self' reference in provisioners?

ANSWER

Within a provisioner block, 'self' refers to the parent resource: provisioner "local-exec" { command = "echo \${self.public_ip}" }. Only allowed inside provisioner blocks. Enables provisioners to access resource attributes without explicit resource type prefix.

Q15 QUESTION

What is an implicit vs explicit dependency?

ANSWER

Implicit: Terraform auto-detects when one resource references another attribute (subnet_id = aws_subnet.main.id). Explicit: manually declared via depends_on when the dependency exists through side effects not captured in attribute references. Prefer implicit; use depends_on sparingly.

Q16 QUESTION

What is 'terraform plan -detailed-exitcode'?

ANSWER

Returns exit code 0 (no changes), 1 (error), or 2 (changes present). Used in CI/CD: terraform plan -detailed-exitcode; if [\$? -eq 2]; then echo Changes pending; fi. Enables automated decision-making and gating based on whether Terraform detects pending infrastructure changes.

Q17 QUESTION

What is a .tftpl template file?

ANSWER

Template files with .tftpl extension contain text with \${var} interpolation, rendered by templatefile() function. Example: templatefile("user_data.tftpl", {region=var.region, env=var.env}). Used for cloud-init scripts, config files, policy documents, and any text with dynamic values.

Q18 QUESTION

What is 'terraform init -backend=false'?

ANSWER

Skips backend initialization. Useful for running terraform validate or generating documentation without backend credentials. State is inaccessible so plan/apply fails, but syntax validation succeeds. Common in CI pipelines that only need to validate code, not apply changes.

Q19 QUESTION

What is the 'any' type constraint?

ANSWER

Variable type 'any' accepts any value without type checking: variable "config" { type = any }. Terraform infers the type at runtime. Useful for flexible module inputs. Best practice: use explicit types (string, list(string), map(string)) for better validation and error messages.

Q20 QUESTION

What are Terraform checks vs validations?

ANSWER

Variable validations (validation block in variable) enforce input constraints before any resource action. Preconditions/postconditions (lifecycle block) validate resource-specific behavior. Check blocks (v1.5+) monitor infrastructure health without blocking. Each serves a different validation layer.

Q21 QUESTION

What is the 'nonsensitive' function?

ANSWER

`nonsensitive(value)` removes the sensitive marking from a value, allowing it to be used in non-sensitive contexts. Use with extreme caution — only when you are certain the value is safe to display. Misuse can expose secrets in logs and plan output.

2 Terraform State Management

17 Qs

Q22 QUESTION

What is Terraform state?

ANSWER

Terraform state (terraform.tfstate) is a JSON file mapping real-world resources to configuration. It stores resource IDs, attributes, metadata, and dependencies. It is the single source of truth for what Terraform manages and enables incremental, efficient updates.

Q23 QUESTION

Why is Terraform state important?

ANSWER

State enables: (1) Resource tracking — maps config to real infrastructure, (2) Metadata storage — IDs, relationships, (3) Incremental updates — only changes delta, (4) Drift detection — compares state vs reality, (5) Team collaboration — shared remote state.

Q24 QUESTION

What is a remote backend?

ANSWER

A remote backend stores state in a shared location (S3, Azure Blob, GCS, Terraform Cloud) instead of locally. Benefits: team collaboration, state locking, encryption, versioning, and audit trails. Essential for any team using Terraform in production.

Q25 QUESTION

What is state locking?

ANSWER

State locking prevents concurrent operations from corrupting the state file. When a Terraform operation runs, the state is locked — other operations must wait. S3 backend uses DynamoDB for locking; Terraform Cloud has built-in locking with queue management.

Q26 QUESTION

How do you view the current Terraform state?

ANSWER

'terraform state list' — lists all resource addresses. 'terraform state show aws_instance.web' — details of a specific resource. 'terraform show' — full state in human-readable format. 'terraform show -json' — machine-readable output for tooling.

Q27 QUESTION

What is 'terraform state mv'?

ANSWER

'terraform state mv' renames a resource in state or moves it to a different module without destroying/recreating. terraform state mv aws_instance.old aws_instance.new. Essential for refactoring configurations while preserving existing infrastructure.

Q28 QUESTION

What is 'terraform state rm'?

ANSWER

'terraform state rm' removes a resource from Terraform state without destroying real infrastructure. After removal, Terraform no longer manages that resource. Use when you want to stop managing a resource or before manually deleting it.

Q29 QUESTION

What is 'terraform import'?

ANSWER

'terraform import' brings existing infrastructure under Terraform management by adding it to state. Steps: (1) Write resource config, (2) Run: `terraform import aws_instance.web i-1234567`. Terraform adds it to state but does not generate HCL — you must write the config.

Q30 QUESTION

What is the import block?

ANSWER

The import block (v1.5+) enables declarative, reviewable imports in configuration: `import { to = aws_instance.web; id = "i-12345" }`. Combined with `-generate-config-out` flag, Terraform can auto-generate the HCL configuration for the imported resource.

Q31 QUESTION

What is state drift?

ANSWER

State drift occurs when actual infrastructure differs from the Terraform state file — caused by manual changes, external tools, or failures. 'terraform plan' detects drift and proposes corrective changes. 'terraform apply -refresh-only' updates state without other changes.

Q32 QUESTION

What is 'terraform state pull' and 'state push'?

ANSWER

'terraform state pull' downloads and prints the current remote state. 'terraform state push' uploads a local state file to the remote backend. Use for backup/restore, emergency fixes, or state migration between backends.

Q33 QUESTION

How do you migrate state between backends?

ANSWER

Steps: (1) Update backend config in terraform block, (2) Run 'terraform init', (3) Terraform detects the change and asks to migrate state, (4) Confirm migration. Always back up state before migration. Verify with 'terraform state list' after.

Q34 QUESTION

What is 'terraform state replace-provider'?

ANSWER

terraform state replace-provider replaces the provider source for resources in state. Used when migrating from a community provider to an official one (e.g., changing `terraform-providers/aws` to `hashicorp/aws`) without destroying resources. Run: `terraform state replace-provider registry.terraform.io/hashicorp/aws`.

Q35 QUESTION

What is state versioning?

ANSWER

State versioning tracks the history of state file changes. Enabled at the backend level: S3 with versioning=true maintains full history. Terraform Cloud stores all run state versions automatically. Enables rollback to previous state if a bad apply corrupts or misconfigures infrastructure.

Q36 QUESTION

What happens when two users run apply simultaneously?

ANSWER

If state locking is enabled (DynamoDB with S3, or Terraform Cloud), the second user gets an error: 'Error locking state: Error acquiring the state lock'. They must wait for the first apply to complete. Without locking, concurrent applies cause state corruption — always use backends with locking.

Q37 QUESTION

What is 'terraform state list' used for?

ANSWER

terraform state list shows all resource addresses currently tracked in state. Output format: module.name.resource_type.resource_name. Useful for verifying what Terraform manages, finding resource addresses for use with -target or state commands, and auditing state contents after import.

Q38 QUESTION

What is the difference between state and plan?

ANSWER

State records what Terraform currently manages (reality as Terraform knows it). Plan is a calculated diff between desired config and current state. State persists between runs; plans are ephemeral. State lives in the backend; saved plans live in binary files. State is ground truth; plan is the proposed change.

3 Terraform Configuration Language

20 Qs

Q39 QUESTION

What are the main block types in Terraform?

ANSWER

Eight main block types: terraform (global settings/backend), provider (cloud credentials/config), resource (infra objects), data (read existing data), variable (input parameters), output (return values), locals (computed values), module (reusable configs).

Q40 QUESTION

What is a Terraform variable?

ANSWER

Variables parameterize configurations. Declared with variable block including type, default, description, validation, and sensitive attributes. Values provided via: .tfvars files, environment variables (TF_VAR_name), CLI flags (-var, -var-file), or interactive prompt.

Q41 QUESTION

What variable types does Terraform support?

ANSWER

Primitive: string, number, bool. Collection: list(type), set(type), map(type). Structural: object({attr=type}), tuple([type]). Special: any (no type constraint). Type constraints prevent misconfiguration and improve error messages.

Q42 QUESTION

What is the order of variable precedence?

ANSWER

Lowest to highest: (1) Default in variable block, (2) terraform.tfvars, (3) *.auto.tfvars alphabetically, (4) -var-file CLI flag, (5) -var CLI flag, (6) TF_VAR_name environment variables. Higher precedence values always override lower ones.

Q43 QUESTION

What is a terraform.tfvars file?

ANSWER

A .tfvars file provides variable values: instance_type = "t3.micro"; region = "us-east-1". Terraform auto-loads terraform.tfvars and *.auto.tfvars. Custom files require -var-file flag. Never commit .tfvars files containing secrets to version control.

Q44 QUESTION

What are Terraform output values?

ANSWER

Outputs expose values after apply: output "ip" { value = aws_instance.web.public_ip; description = "Public IP"; sensitive = false }. Shown after apply, accessible via 'terraform output', and used by other modules or scripts consuming Terraform state.

Q45 QUESTION

What are local values?

ANSWER

Locals compute reusable values within a module: locals { full_name = "\${var.env}-\${var.app}" }. Referenced as local.full_name. Useful for reducing repetition, computing complex values once, and improving readability without exposing as module inputs.

Q46 QUESTION

What is the 'count' meta-argument?

ANSWER

'count' creates N identical resource instances: count = 3 creates [0], [1], [2]. Use count.index to differentiate. Remove a middle instance and Terraform must shift indexes, potentially destroying and recreating instances — this is a key limitation versus for_each.

Q47 QUESTION

What is the 'for_each' meta-argument?

ANSWER

'for_each' creates instances from a map or set with stable identifiers: for_each = {"dev"="t2.micro", "prod"="t3.large"}. Access via each.key and each.value. Adding/removing one entry only affects that entry — no index shifting. Preferred over count for named resources.

Q48 QUESTION

What is a dynamic block?

ANSWER

Dynamic blocks generate repeated nested configuration blocks from a collection: dynamic "ingress" { for_each = var.rules; content { from_port = ingress.value.port; cidr_blocks = ingress.value.cidrs } }. Avoids copy-paste of repeated nested blocks.

Q49 QUESTION

What is a conditional expression?

ANSWER

Conditional expressions implement ternary logic: condition ? true_val : false_val. Example: instance_type = var.env == "prod" ? "t3.large" : "t2.micro". Used inline in arguments to vary configuration based on variables.

Q50 QUESTION

What is a 'for' expression?

ANSWER

For expressions transform collections: [for s in list : upper(s)] — new list; { for k,v in map : k => v*2 } — new map; [for s in list : s if s != ""] — filtered list. Essential for data transformation in Terraform configurations.

Q51 QUESTION

What is a variable validation block?

ANSWER

Validation blocks enforce input constraints: variable "instance_type" { validation { condition = contains(["t3.micro", "t3.small"], var.instance_type); error_message = "Must be t3.micro or t3.small." } }. Terraform checks conditions before plan and surfaces clear, actionable error messages.

Q52 QUESTION

What is the 'null' value in Terraform?

ANSWER

null represents the absence of a value. Passing null for an optional argument uses the default value: instance_type = var.override != null ? var.override : "t3.micro". null is useful for conditionally omitting optional block arguments and resetting attributes to provider defaults.

Q53 QUESTION

What is a heredoc string in Terraform?

ANSWER

Heredoc strings define multi-line values using the syntax: `policy = <<EOT ... EOT`. Use `<<-EOT` (with dash) for indented heredocs that strip leading whitespace. Heredocs are useful for embedding JSON, YAML, scripts, and policy documents directly in configuration.

Q54 QUESTION

What is the 'one' function in Terraform?

ANSWER

`one(list)` returns the only element if the list has one item, or null if empty, and errors if the list has 2+ elements: `one(aws_eip.main[*].id)`. Useful with conditional count resources to safely get the single instance value or null without index errors.

Q55 QUESTION

What is 'setunion', 'setintersection', 'setsubtract'?

ANSWER

Set operations: `setunion([a,b],[b,c]) = [a,b,c]` (all unique elements); `setintersection([a,b],[b,c]) = [b]` (common elements); `setsubtract([a,b,c],[b]) = [a,c]` (elements in first but not second). Useful for computing differences in security group rules, tags, or any set-based config.

Q56 QUESTION

What is 'can' function in Terraform?

ANSWER

`can(expression)` returns true if the expression evaluates without errors, false otherwise: `can(regex("^prod-", var.name))`. Useful inside validation conditions to safely test expressions that might fail. Often paired with `try()` for defensive configuration patterns.

Q57 QUESTION

What is 'try' function in Terraform?

ANSWER

`try(expr1, expr2, fallback)` evaluates expressions in order and returns the first that doesn't produce an error: `try(var.config.timeout, 30)`. Useful for handling optional nested attributes in objects where a missing key would otherwise cause an error.

Q58 QUESTION

What are object and tuple types?

ANSWER

`object({attr=type})` defines a structural type with named attributes: variable `"server" { type = object({name=string; count=number}) }`. `tuple([type, type])` defines an ordered sequence of specific types. Both enforce structure — unlike `map`/`list` which require uniform element types.

4 Plan, Apply & Lifecycle

17 Qs

Q59 QUESTION

What does 'terraform plan' do?

ANSWER

'terraform plan' creates a safe, read-only execution plan showing exactly what changes will be made. Output uses + (create), ~ (update in-place), - (destroy), -/+ (destroy and recreate). Must-run before every apply in production environments.

Q60 QUESTION

What does 'terraform apply' do?

ANSWER

'terraform apply' executes changes to reach the desired state — creating, modifying, or destroying resources. Runs plan first and prompts for confirmation. Use -auto-approve in CI/CD. Use -var, -var-file, and -target flags as needed. State is updated after each apply.

Q61 QUESTION

What does 'terraform destroy' do?

ANSWER

'terraform destroy' removes ALL resources managed by the current configuration. It generates a destroy plan for review before execution. Equivalent to 'terraform apply -destroy'. Use -target=resource_address to destroy specific resources only.

Q62 QUESTION

What is the '-target' flag?

ANSWER

'-target=resource_address' limits plan/apply to specific resources: terraform apply -target=aws_instance.web. Useful for testing or emergency fixes. Use sparingly — targeted applies can leave state inconsistent and mask configuration issues.

Q63 QUESTION

What is 'create_before_destroy'?

ANSWER

lifecycle { create_before_destroy = true } instructs Terraform to create the replacement resource before destroying the old one. Critical for zero-downtime updates on resources like load balancers, certificates, or any resource where brief unavailability is unacceptable.

Q64 QUESTION

What is 'prevent_destroy'?

ANSWER

lifecycle { prevent_destroy = true } causes Terraform to error if any plan would destroy that resource. Essential safeguard for databases, S3 buckets with data, and other critical resources. The error forces explicit configuration change to remove this protection.

Q65 QUESTION

What is 'ignore_changes'?

ANSWER

`lifecycle { ignore_changes = [tags, ami] }` tells Terraform to ignore changes to listed attributes, preventing perpetual plan diffs for attributes managed externally. Use 'all' to ignore all changes and make Terraform only create/destroy the resource.

Q66 QUESTION

What is 'replace_triggered_by'?

ANSWER

`lifecycle { replace_triggered_by = [aws_iam_role.example] }` forces resource replacement when the referenced resource changes. This creates explicit replacement chains for resources that have hidden dependencies not captured by attribute references.

Q67 QUESTION

What is 'terraform plan -out'?

ANSWER

'terraform plan -out=tfplan' saves the execution plan to a binary file. Apply it exactly: 'terraform apply tfplan'. This guarantees CI/CD applies exactly what was reviewed and approved — the plan cannot change between plan and apply stages.

Q68 QUESTION

What is '-refresh-only' mode?

ANSWER

'terraform plan/apply -refresh-only' updates the state file to match real infrastructure without making other changes. It allows reviewing and accepting drift. Replaces the deprecated 'terraform refresh' command with a safer, reviewable workflow.

Q69 QUESTION

What are the resource change types in plan output?

ANSWER

+ (add/create): new resource will be created. ~ (update): existing resource will be updated in-place. - (destroy): resource will be deleted. -/+ (replace): destroy old and create new (for immutable attributes). +/- (replace): create before destroy variant.

Q70 QUESTION

What happens during 'terraform apply'?

ANSWER

Terraform: (1) Runs a fresh plan, (2) Shows the changes, (3) Prompts for confirmation, (4) Builds the dependency graph, (5) Executes creates/updates/destroys in dependency order, (6) Runs parallel where possible, (7) Updates state file after each resource, (8) Shows summary and outputs.

Q71 QUESTION

What is replace_triggered_by?

ANSWER

`lifecycle { replace_triggered_by = [aws_iam_role.app] }` forces resource replacement when the referenced resource changes — even if the resource itself has no changes. Creates explicit replacement chains for hidden dependencies. References can be resources, modules, or specific resource attributes.

Q72 QUESTION

What is the difference between update in-place and replace?

ANSWER

Update in-place (~): Terraform calls the provider's Update API to modify the existing resource. Replace (-/+): the old resource is destroyed and a new one is created because the changed attribute is 'ForceNew' (immutable). Replace causes downtime unless `create_before_destroy` is used.

Q73 QUESTION

What does Terraform do with unrecognized attributes in state?

ANSWER

Unrecognized attributes in state (from an older provider version, or removed from the provider schema) are preserved but ignored during plan/apply. They may cause provider errors if the provider rejects unknown fields. Upgrading or downgrading providers can surface these issues.

Q74 QUESTION

What is 'terraform apply -var' used for?

ANSWER

'-var name=value' sets a variable value on the command line: `terraform apply -var=env=prod -var=region=us-east-1`. Multiple -var flags allowed. Command-line values have the second-highest precedence after `TF_VAR_` environment variables. Used for passing dynamic values in CI/CD pipelines.

Q75 QUESTION

What is 'terraform apply -parallelism'?

ANSWER

'-parallelism=N' controls how many resource operations Terraform performs concurrently (default: 10). Increase for faster applies with many independent resources. Decrease if hitting API rate limits. Set to 1 to force completely sequential execution for debugging race conditions.

5 Terraform Modules

15 Qs

Q76 QUESTION

What is a Terraform module?

ANSWER

A module is a container of resources used together. The root module is the working directory. Child modules are called via the 'module' block and can be sourced locally, from Git, or the Terraform Registry. Modules enable DRY code, encapsulation, and versioned infrastructure patterns.

Q77 QUESTION

How do you call a module?

ANSWER

Use the module block: `module "vpc" { source = "../modules/vpc"; version = "~>3.0"; cidr_block = var.cidr }`. Source can be a local path, Git URL (`github.com/org/repo`), Terraform Registry (`terraform-aws-modules/vpc/aws`), or archive URL.

Q78 QUESTION

What is the Terraform Module Registry?

ANSWER

The public registry at `registry.terraform.io` hosts verified and community modules. Sourced as: `source = "terraform-aws-modules/vpc/aws"`. Verified modules (blue badge) are maintained by trusted partners. Version constraints apply just like providers.

Q79 QUESTION

What are module inputs and outputs?

ANSWER

Inputs are variables declared in the module the caller provides. Outputs are values the module exposes: accessible as `module..` in the parent. This enables composition — passing VPC ID from a networking module to a compute module.

Q80 QUESTION

What is the private module registry?

ANSWER

Terraform Cloud/Enterprise hosts a private registry for internal modules. Follows same naming: `///`. Supports versioning, documentation, and access controls. Allows organizations to share vetted modules securely across teams.

Q81 QUESTION

What is module versioning?

ANSWER

Version constraints specify acceptable module versions: `version = "~> 3.0" (>= 3.0, < 4.0)`. Operators: `=`, `!=`, `>`, `>=`, `<`, `<=`, `~>`. Pin to specific version in production. Use `'terraform init -upgrade'` to update modules within constraints.

Q82 QUESTION

Can modules call other modules?

ANSWER

Yes — nested modules are supported. A child module can call another child module. Best practice: limit nesting to 2 levels. Deep nesting increases complexity, makes debugging harder, and reduces reusability. Prefer flat module composition over deep hierarchies.

Q83 QUESTION

What is module composition?

ANSWER

Module composition assembles infrastructure from small, focused modules. Each module owns one concern (networking, compute, security). The root module composes them by passing outputs as inputs: `module.vpc.vpc_id` fed to `module.compute`. Enables flexible, testable architectures.

Q84 QUESTION

What does 'terraform get' do?

ANSWER

'terraform get' downloads and updates modules referenced in configuration to `.terraform/modules/`. Automatically run by 'terraform init'. Use 'terraform get -update' to check for newer versions. Not needed manually in most workflows.

Q85 QUESTION

What is published module naming convention?

ANSWER

Public registry modules follow: `//` — e.g., `hashicorp/consul/aws`. The provider suffix indicates which cloud/platform the module targets. The same module name can exist for multiple providers (aws, azure, gcp).

Q86 QUESTION

What is the 'source' argument in a module block?

ANSWER

The source argument specifies where to find module code: local path (`./modules/vpc`), Terraform Registry (`hashicorp/consul/aws`), GitHub (`github.com/org/repo`), generic Git (`git::https://...`), Bitbucket, S3 bucket, or GCS bucket. Local paths start with `./` or `../`; all others are downloaded.

Q87 QUESTION

What is module pinning best practice?

ANSWER

Always pin module versions in production: `module "vpc" { source="terraform-aws-modules/vpc/aws"; version="5.1.2" }`. Unpinned modules (no version) always use the latest, which can introduce breaking changes. Pin to exact version or `~>` constraint. Update deliberately after testing.

Q88 QUESTION

What is a `modules.json` file?

ANSWER

`.terraform/modules/modules.json` is generated by `terraform init` and records downloaded module sources, versions, and local paths. It maps module call addresses to their cache locations. Do not commit to version control — it is environment-specific and regenerated by `init`.

Q89 QUESTION

What are module outputs best practices?

ANSWER

Output everything callers might need: resource IDs, ARNs, endpoints, security group IDs. Use descriptive names: output "vpc_id" not "id". Add description to every output. Mark sensitive=true for passwords/keys. Avoid outputting the entire resource object — output specific attributes.

Q90 QUESTION

What is a generic module pattern?

ANSWER

A generic module accepts inputs, creates resources, and exposes outputs. Pattern: variables.tf (all inputs), main.tf (resources), outputs.tf (all outputs), versions.tf (required_providers/required_version). Optional: README.md, examples/, tests/. This structure enables reuse and discoverability.

6 Backends & Remote State

8 Qs

Q91 QUESTION

What is a Terraform backend?

ANSWER

A backend defines where state is stored and optionally where operations run. Configured inside the terraform block. Default is 'local' (disk). Remote backends (s3, azurerm, gcs, Terraform Cloud) enable team collaboration, locking, and encryption.

Q92 QUESTION

What is the S3 backend?

ANSWER

The S3 backend stores state in AWS S3. Config: terraform { backend "s3" { bucket="tfstate-prod"; key="prod/terraform.tfstate"; region="us-east-1"; encrypt=true; dynamodb_table="tf-locks" } }. DynamoDB table enables state locking. Supports versioning for state history.

Q93 QUESTION

What is partial backend configuration?

ANSWER

Partial config provides some backend settings in code and the rest via -backend-config flag or file: terraform init -backend-config=backend.hcl. Enables keeping secrets (access keys, tokens) out of version control while still defining backend type in code.

Q94 QUESTION

What is the 'terraform_remote_state' data source?

ANSWER

Reads outputs from another Terraform state: data "terraform_remote_state" "vpc" { backend="s3"; config={bucket="tfstate"; key="vpc.tfstate"} }. Access outputs as: data.terraform_remote_state.vpc.outputs.vpc_id. Enables cross-configuration data sharing.

Q95 QUESTION

How do you enable state encryption?

ANSWER

S3 backend: set encrypt=true and optionally kms_key_id for KMS encryption. Terraform Cloud encrypts state at rest and in transit automatically. Local state is NOT encrypted — never store sensitive values in local state files in production.

Q96 QUESTION

What is 'terraform state pull/push'?

ANSWER

'terraform state pull' downloads current remote state to stdout. 'terraform state push' uploads a local file to the remote backend. Used for state backup, emergency recovery, or manual state surgery. Always backup before pushing.

Q97 QUESTION

What is Terraform Cloud as a backend?

ANSWER

Use the 'remote' or 'cloud' backend block: `terraform { cloud { organization="myorg"; workspaces { name="prod" } } }`. Terraform Cloud stores state, runs operations remotely, provides UI, audit logs, team permissions, and policy enforcement.

Q98 QUESTION

What is the 'cloud' backend block?

ANSWER

The 'cloud' block (v1.1+) is the modern way to connect to HCP Terraform/Terraform Cloud: `terraform { cloud { organization = "myorg"; workspaces { tags = ["prod"] } } }`. Preferred over the legacy 'remote' backend block.

7 Terraform Workspaces

5 Qs

Q99 QUESTION

What are Terraform workspaces?

ANSWER

Workspaces are named state files within the same backend. Each workspace has independent state, enabling environment separation (dev/staging/prod) with the same configuration. Default workspace is 'default'. State stored in terraform.tfstate.d/./.

Q100 QUESTION

What are the workspace commands?

ANSWER

terraform workspace new — create; terraform workspace list — show all; terraform workspace select — switch; terraform workspace show — current name; terraform workspace delete — remove (must be empty, non-default).

Q101 QUESTION

How do you reference the current workspace in config?

ANSWER

Use terraform.workspace built-in: instance_type = terraform.workspace == "prod" ? "t3.large" : "t2.micro". Also useful in resource names, tags, and variable selection. Enables single config to behave differently per environment.

Q102 QUESTION

When to use workspaces vs separate configurations?

ANSWER

Use workspaces for similar environments (dev/prod same structure, minor differences). Use separate directories/repos when: environments have drastically different infra, different teams own them, you need strict access isolation, or environments have different provider configs.

Q103 QUESTION

What is the difference between OSS and TF Cloud workspaces?

ANSWER

OSS workspaces: state isolation within one backend, same variables/config for all. TF Cloud workspaces: full isolation — each has own state, variables, run history, policies, teams, notifications. TF Cloud workspaces map to individual configurations, not just state files.

8 Built-in Functions

10 Qs

Q104 QUESTION

What function categories exist in Terraform?

ANSWER

Numeric (abs, ceil, floor, max, min, pow), String (format, join, split, replace, trim, upper, lower, substr, regex), Collection (concat, flatten, length, lookup, merge, toset, tolist, tomap, keys, values, contains), Encoding (jsonencode, jsondecode, base64encode, yamlencode), Filesystem (file, filebase64), Date/Time, IP Network (cidrsubnet, cidrhost), Type Conversion.

Q105 QUESTION

What does 'lookup' do?

ANSWER

lookup(map, key, default) retrieves a map value by key, returning default if missing: lookup(var.amis, var.region, "ami-default"). Use to select environment-specific values from a map without errors when a key might not exist.

Q106 QUESTION

What does 'merge' do?

ANSWER

merge(map1, map2, ...) combines maps — later maps override earlier for duplicate keys: local.all_tags = merge(var.common_tags, var.resource_tags). Essential for combining default tags with resource-specific tags across all resources.

Q107 QUESTION

What does 'flatten' do?

ANSWER

flatten(nested_list) collapses nested lists into one flat list: flatten(["a","b"],["c"]) = ["a","b","c"]. Used when combining lists from multiple module outputs or when for_each/count produces nested lists that need to be iterable.

Q108 QUESTION

What does 'cidrsubnet' do?

ANSWER

cidrsubnet(prefix, newbits, netnum) calculates subnet addresses: cidrsubnet("10.0.0.0/16", 8, 0) = "10.0.0.0/24"; cidrsubnet("10.0.0.0/16", 8, 1) = "10.0.1.0/24". Automates subnet CIDR calculation from a parent CIDR, eliminating manual calculation.

Q109 QUESTION

What does 'templatefile' do?

ANSWER

templatefile(path, vars) reads a file and renders it as a template with provided variables: templatefile("user_data.tpl", {region=var.region, env=var.env}). The template uses \${var_name} syntax. Replaces the deprecated template_file data source.

Q110 QUESTION

What does 'toset' do?

ANSWER

`toset(list)` converts a list to a set — removes duplicates, loses ordering. Commonly used with `for_each`: `for_each = toset(["us-east-1", "us-west-2"])`. Sets cannot contain null values. Converting to set before `for_each` prevents duplicate instance creation.

Q111 QUESTION

What does the splat expression (*) do?

ANSWER

The splat expression extracts an attribute from all instances: `aws_instance.web[*].id` is equivalent to `[for o in aws_instance.web : o.id]`. Works on lists and count-based resources. Returns a list of attribute values from all instances.

Q112 QUESTION

What does 'coalesce' do?

ANSWER

`coalesce(val1, val2, ...)` returns the first non-null, non-empty-string value. `coalescelist` returns the first non-empty list. Useful for fallback chains: `coalesce(var.custom_name, local.default_name, "fallback")`.

Q113 QUESTION

What does 'jsonencode' / 'jsondecode' do?

ANSWER

`jsonencode(value)` converts a Terraform value to a JSON string. `jsondecode(string)` parses JSON into a Terraform value. Commonly used for policy documents, `user_data`, and any API that requires JSON string inputs.

9 Security & Best Practices

8 Qs

Q114 QUESTION

How do you manage secrets in Terraform?

ANSWER

Best practices: (1) Use env vars (TF_VAR_name) not hardcoded values, (2) Use secret managers (Vault, AWS Secrets Manager, Azure Key Vault), (3) Mark variables/outputs as sensitive=true, (4) Encrypt remote state, (5) Never commit .tfvars with secrets, (6) Use dynamic credentials where possible.

Q115 QUESTION

What is the 'sensitive' attribute?

ANSWER

sensitive=true on variables or outputs redacts values in CLI output and logs: variable "db_password" { type=string; sensitive=true }. IMPORTANT: sensitive values are still stored in plaintext in the state file – use external secret management for true protection.

Q116 QUESTION

What is the .terraform.lock.hcl file?

ANSWER

The dependency lock file records exact provider versions and checksums selected by 'terraform init'. Ensures all team members and CI systems use identical provider versions. Commit to version control. Use 'terraform init -upgrade' to update locked versions within constraints.

Q117 QUESTION

Why pin provider versions?

ANSWER

Pinning prevents breaking changes from new provider releases: required_providers { aws = { source="hashicorp/aws"; version="~>5.0" } }. The ~> operator (pessimistic constraint) allows patch/minor updates but blocks major version upgrades. Always pin in production.

Q118 QUESTION

What is 'terraform taint' (deprecated)?

ANSWER

'terraform taint' marked a resource for forced replacement. Deprecated in v0.15. Replaced by: terraform apply -replace=aws_instance.web. The -replace flag shows the replacement in plan output and prompts for confirmation – safer and more explicit.

Q119 QUESTION

What are Terraform team collaboration best practices?

ANSWER

(1) Remote state with locking, (2) Git for version control with PR reviews, (3) Workspaces or separate dirs per environment, (4) Pin provider versions, (5) Commit lock file, (6) Use Terraform Cloud or CI/CD for automated runs, (7) Never run apply locally in production.

Q120 QUESTION

What is 'terraform graph'?

ANSWER

'terraform graph' generates a DOT-format dependency graph. Visualize with Graphviz: `terraform graph | dot -Tsvg > graph.svg`. Shows resource dependencies, execution order, and parallelism opportunities. Use `-type` flag for plan, apply, or destroy graph variants.

Q121 QUESTION

What is 'terraform console'?

ANSWER

'terraform console' opens an interactive REPL for evaluating expressions against current state and config. Test functions: `> cidrsubnet("10.0.0.0/16", 8, 1)` returns "10.0.1.0/24". Test references and complex expressions before adding to configuration.

10 Terraform Cloud & Enterprise

10 Qs

Q122 QUESTION

What is HCP Terraform (Terraform Cloud)?

ANSWER

HCP Terraform (formerly Terraform Cloud) is HashiCorp's managed SaaS platform providing: remote state storage, remote plan/apply runs, team collaboration, Sentinel policy enforcement, private module registry, audit logging, cost estimation, SSO, and workspace management.

Q123 QUESTION

What is Terraform Enterprise?

ANSWER

Terraform Enterprise is the self-hosted version of HCP Terraform for organizations with strict compliance needs. Additional features: SAML/LDAP SSO, HSM support, audit log streaming, private network installation, clustering for HA, and no resource/run limits.

Q124 QUESTION

What is Sentinel policy as code?

ANSWER

Sentinel is HashiCorp's policy enforcement framework in Terraform Cloud/Enterprise. Policies check runs before apply: verify resource types, naming conventions, cost limits, security settings, or mandatory tags. Written in Sentinel language, applied via policy sets to workspaces.

Q125 QUESTION

What are Sentinel enforcement levels?

ANSWER

(1) Advisory: policy failure logged but run proceeds, (2) Soft Mandatory: blocks run but authorized users can override, (3) Hard Mandatory: always blocks non-compliant runs — cannot be overridden regardless of permissions. Most security policies use Hard Mandatory.

Q126 QUESTION

What is a Terraform Cloud workspace?

ANSWER

A TF Cloud workspace is the primary unit of infrastructure management — each has own state, variables (including sensitive env vars), run history, notifications, permissions, and settings. Can be VCS-connected (auto-runs on commits) or API/CLI-driven.

Q127 QUESTION

What are run triggers in Terraform Cloud?

ANSWER

Run triggers automatically start a new plan/apply in workspace B when workspace A completes successfully. This models infrastructure dependencies: networking workspace finishes → compute workspace triggers. Replaces manual dependency management between stacks.

Q128 QUESTION

What is cost estimation in Terraform Cloud?

ANSWER

Before apply, Terraform Cloud shows estimated monthly infrastructure cost based on the planned changes using cloud provider pricing data. Available in Team & Governance and Business tiers. Helps catch costly misconfigurations (oversized instances, unintended resources) early.

Q129 QUESTION

What are Terraform Cloud execution modes?

ANSWER

Remote: plans and applies run on TF Cloud infrastructure — best for teams. Local: operations run on local machine, state stored in TF Cloud. Agent: operations run on a private agent pool inside your network — for accessing private/on-premises resources not reachable from internet.

Q130 QUESTION

What is a Terraform Cloud agent?

ANSWER

Agents (Business tier) are lightweight processes that run inside your private network and execute Terraform operations on behalf of Terraform Cloud. Agent pools group multiple agents for availability. Enable TF Cloud to reach resources not accessible from the public internet.

Q131 QUESTION

What is the private module registry?

ANSWER

Organizations publish internal modules to the private registry in Terraform Cloud/Enterprise. Modules are versioned, documented, and access-controlled. Team members source them exactly like public registry modules. Enables governance of approved infrastructure patterns.

11 Advanced Concepts

10 Qs

Q132 QUESTION

What is the Terraform dependency graph?

ANSWER

Terraform builds a DAG (Directed Acyclic Graph) from explicit references (resource A uses resource B's output → A depends on B) and `depends_on` declarations. The graph determines execution order – independent resources run in parallel, dependents wait. Use 'terraform graph' to visualize.

Q133 QUESTION

What is 'depends_on'?

ANSWER

'depends_on' creates explicit dependencies when Terraform cannot infer them from references – hidden dependencies through side effects: resource "aws_instance" "app" { depends_on = [aws_iam_role_policy.app] }. Use sparingly; explicit references are preferred.

Q134 QUESTION

What is a moved block?

ANSWER

Moved blocks (v1.1+) enable state refactoring without destroying resources: `moved { from = aws_instance.web; to = module.compute.aws_instance.web }`. Terraform updates state addresses without touching infrastructure. Remove moved blocks after applying and verify with plan.

Q135 QUESTION

What are preconditions and postconditions?

ANSWER

Custom conditions (v1.2+) validate resource behavior with clear error messages. `lifecycle { precondition { condition = var.instance_type != "t1.micro"; error_message = "t1.micro is end-of-life" } }`. Preconditions check before, postconditions check after the resource action.

Q136 QUESTION

What is the check block?

ANSWER

Check blocks (v1.5+) define infrastructure health assertions that run every plan/apply but never block: `check "alb_healthy" { assert { condition = aws_lb.main.state == "active"; error_message = "Load balancer not active" } }`. Failures emit warnings, not errors.

Q137 QUESTION

What is the 'terraform_data' resource?

ANSWER

'terraform_data' (v1.4+) is a built-in resource replacing `null_resource`. Stores arbitrary values in state and supports `triggers_replace` for forcing resource replacement. No provider plugin required. Use for running provisioners or as dependency anchors.

Q138 QUESTION

What is CDKTF?

ANSWER

Cloud Development Kit for Terraform (CDKTF) lets you define Terraform infrastructure using TypeScript, Python, Java, C#, or Go. CDK code compiles to standard JSON Terraform configuration. Enables use of full programming language features (loops, classes, inheritance) for infrastructure.

Q139 QUESTION

What is 'terraform test'?

ANSWER

'terraform test' (v1.6+) runs .tftest.hcl files to validate modules. Tests can execute plan or apply, assert specific values, and clean up after execution. Enables unit and integration testing of Terraform modules as part of CI/CD pipelines.

Q140 QUESTION

What is generated configuration import?

ANSWER

`terraform plan -generate-config-out=generated.tf` (v1.5+) combined with import blocks automatically generates HCL configuration for resources being imported. Eliminates manual config writing for existing resources. Review and adjust generated code before committing.

Q141 QUESTION

What is the '.terraform' directory?

ANSWER

Created by 'terraform init', it contains: provider plugins (.terraform/providers/), downloaded modules (.terraform/modules/), and backend metadata (.terraform/terraform.tfstate). Add to .gitignore — it is environment-specific and should be regenerated via 'terraform init'.

12 Terraform CLI Commands

8 Qs

Q142 QUESTION

What does 'terraform init -upgrade' do?

ANSWER

'terraform init -upgrade' updates all providers and modules to the latest versions within version constraints and updates the .terraform.lock.hcl file. Without -upgrade, init uses existing locked versions. Use regularly to get security fixes and new features.

Q143 QUESTION

What is 'terraform output'?

ANSWER

'terraform output' displays all root module outputs after apply. 'terraform output ' shows a specific value. '-json' flag for machine-readable output. Useful for extracting values (IPs, ARNs) for use in scripts, pipelines, or downstream configurations.

Q144 QUESTION

What does 'terraform providers' do?

ANSWER

'terraform providers' prints providers required by current config. 'terraform providers lock' updates lock file for specified platforms. 'terraform providers mirror ' copies provider binaries to a local directory for air-gapped/offline installations.

Q145 QUESTION

What is 'terraform force-unlock'?

ANSWER

'terraform force-unlock ' manually releases a stuck state lock. Use ONLY when certain no other operation is running. Forcibly unlocking an active operation corrupts state. Get the lock ID from the error message that appears during a locked state operation.

Q146 QUESTION

What is 'terraform login'?

ANSWER

'terraform login' authenticates with Terraform Cloud/Enterprise and stores credentials in ~/.terraform.d/credentials.tfrc.json for use with the cloud backend. 'terraform logout' removes stored credentials. Required before using cloud-hosted state or running remote operations.

Q147 QUESTION

What does 'terraform version' show?

ANSWER

'terraform version' displays CLI version and all installed provider versions. '-json' flag for machine-readable output. Use in troubleshooting to confirm version compatibility. Use 'required_version' in config to enforce minimum CLI versions across teams.

Q148 QUESTION

What is 'terraform apply -json'?

ANSWER

'-json' outputs plan and apply progress as machine-readable JSON to stdout. Enables programmatic parsing of Terraform operations in CI/CD pipelines, custom tooling, and audit systems. Each line is a JSON object describing a specific event or change.

Q149 QUESTION

What is 'terraform workspace' command?

ANSWER

Full workspace management: new (create), select (switch), list (show all with * on current), show (print current name), delete (remove empty non-default workspace). The current workspace is available in config via the terraform.workspace interpolation.

13 Exam Preparation (Terraform 003)

10 Qs

Q150 QUESTION

What is the Terraform Associate (003) exam format?

ANSWER

57 questions (multiple choice and multiple select), 60 minutes, online proctored. No hands-on lab. Passing score approximately 70%. Topics: IaC concepts, Terraform basics, HCL syntax, modules, workflow, state management, Terraform Cloud, and HCP Terraform features.

Q151 QUESTION

What are the main exam domains?

ANSWER

(1) Understand IaC concepts, (2) Understand Terraform purpose and use cases, (3) Understand Terraform basics, (4) Use HCL syntax, (5) Interact with modules, (6) Navigate Terraform workflow, (7) Implement/maintain state, (8) Read/generate/modify configuration, (9) Understand HCP Terraform capabilities.

Q152 QUESTION

What file extensions should you know for the exam?

ANSWER

.tf — HCL configuration files; .tf.json — JSON format config; .tfvars — variable definition files (auto-loaded); .tfvars.json — JSON variable files; .tftest.hcl — test files; terraform.tfstate — state file (JSON). Lock file: .terraform.lock.hcl.

Q153 QUESTION

What is the difference between plan and apply for the exam?

ANSWER

Plan is read-only and safe — previews changes, never modifies real infrastructure, can be run anytime. Apply modifies real infrastructure — runs a plan, prompts for confirmation, updates state after each resource. -auto-approve skips the prompt. Both read current state before running.

Q154 QUESTION

What distinguishes Terraform OSS from Enterprise?

ANSWER

OSS: free, CLI-based, local or remote state, community support, workspaces, no policy enforcement. Enterprise adds: SAML/SSO, LDAP, HSM, audit log streaming, Sentinel policies, private network install, clustering/HA, SLA support. TF Cloud adds most features as managed SaaS.

Q155 QUESTION

What is HCP Terraform?

ANSWER

HCP Terraform is the new name (2024) for Terraform Cloud, now part of the HashiCorp Cloud Platform. Accessible at app.terraform.io. Provides managed Terraform runs, remote state, team workspaces, Sentinel policies, private registry, audit logs, and cost estimation.

Q156 QUESTION

What are key exam tips for Terraform Associate?

ANSWER

(1) Master state management and remote backends, (2) Know all CLI commands, (3) Know variable precedence order, (4) Understand count vs for_each differences, (5) Know module sources (local, registry, git), (6) Know lifecycle arguments, (7) Understand TF Cloud features vs OSS, (8) Practice 'terraform plan' output interpretation.

Q157 QUESTION

What sensitive data considerations are tested?

ANSWER

Variables/outputs with sensitive=true are redacted in CLI output but stored plaintext in state. State files containing sensitive data must use encrypted remote backends. Never store credentials in .tf files. Use environment variables or secret managers. Sentinel can enforce security policies.

Q158 QUESTION

What are the main Terraform Cloud tiers?

ANSWER

Free: up to 500 resources, 1 concurrent run, remote state, basic runs. Plus: unlimited resources, SSO, audit logs, self-hosted agents, run tasks. Business: unlimited, multiple concurrent runs, advanced policies. HCP Terraform Enterprise: self-hosted option with compliance features.

Q159 QUESTION

What is the difference between workspaces (OSS vs Cloud)?

ANSWER

OSS workspaces: state namespacing within a single backend. Same provider config, variables, and code for all workspaces — only state differs. Terraform Cloud workspaces: full isolation with own state, variables (including sensitive), run history, permissions, notifications. Much more powerful.

14 Provisioners & Connections

10 Qs

Q160 QUESTION

What are Terraform provisioners?

ANSWER

Provisioners execute scripts on local or remote machines during resource creation or destruction. Types: local-exec (runs locally on Terraform machine), remote-exec (runs on the created resource via SSH/WinRM), file (copies files to resource). Use as last resort — prefer cloud-init or configuration management tools.

Q161 QUESTION

What is the local-exec provisioner?

ANSWER

local-exec runs a command on the Terraform machine: `provisioner "local-exec" { command = "echo ${self.private_ip} >> ip_list.txt" }`. Runs in the local shell. Use interpreter argument to specify shell. Useful for triggering external scripts, APIs, or updating DNS/inventory systems.

Q162 QUESTION

What is the remote-exec provisioner?

ANSWER

remote-exec runs commands on the remote resource over SSH/WinRM: `provisioner "remote-exec" { inline = ["sudo apt-get update", "sudo apt-get install -y nginx"] }`. Requires a connection block. Useful for bootstrapping software when `user_data/cloud-init` is insufficient.

Q163 QUESTION

What is the file provisioner?

ANSWER

The file provisioner copies files/directories from local to remote: `provisioner "file" { source = "config.cfg"; destination = "/etc/app/config.cfg" }`. Requires a connection block. Often combined with remote-exec to copy then execute scripts.

Q164 QUESTION

What is the connection block?

ANSWER

The connection block specifies how to connect to a remote resource: `connection { type="ssh"; host=self.public_ip; user="ubuntu"; private_key=file("~/ssh/id_rsa") }`. Types: ssh (Linux) and winrm (Windows). Can be defined at resource or provisioner level.

Q165 QUESTION

What happens when a provisioner fails?

ANSWER

By default (`on_failure = fail`), Terraform marks the resource as tainted and errors. On next apply, the tainted resource is destroyed and recreated. Set `on_failure = continue` to ignore failures. Provisioner failures are a common source of partial state issues.

Q166 QUESTION

What is when = destroy for provisioners?

ANSWER

Provisioners with when = destroy run before the resource is destroyed: provisioner "local-exec" { when = destroy; command = "./deregister.sh \${self.id}" }. Used for graceful deregistration, cleanup tasks, or notifying external systems before teardown.

Q167 QUESTION

Why should provisioners be avoided?

ANSWER

Provisioners add complexity, break idempotency, can fail, require network access, and depend on external scripts. They make terraform plan output incomplete. Prefer: user_data for EC2 bootstrap, pre-baked AMIs, cloud-init, or configuration management tools like Ansible.

Q168 QUESTION

What is terraform_data resource?

ANSWER

terraform_data (v1.4+ built-in) replaces null_resource without needing a provider. It stores arbitrary values in state and supports triggers_replace to force replacement: resource "terraform_data" "bootstrap" { triggers_replace = [aws_instance.web.id] }. Used to run provisioners or as dependency anchors.

Q169 QUESTION

What is the order of provisioner execution?

ANSWER

Multiple provisioners on a resource execute in order of declaration, top to bottom. Creation-time provisioners (default) run after creation. Destroy-time provisioners (when=destroy) run before destruction. If any provisioner fails, subsequent provisioners on the same resource are skipped.

15 Terraform Registry & Providers

10 Qs

Q170 QUESTION

What is the Terraform Registry?

ANSWER

The Terraform Registry (registry.terraform.io) is the central hub for public providers and modules. Providers are published by HashiCorp, cloud vendors, and the community. Verified providers and modules have a badge indicating official or partner status. Source address format: `hostname/namespace/type`.

Q171 QUESTION

What are Official vs Partner vs Community providers?

ANSWER

Official: developed and supported by HashiCorp (aws, azure, google, kubernetes). Partner: maintained by technology partners verified by HashiCorp (datadog, mongodb, cloudflare). Community: maintained by individuals – quality and support varies. All are on the public registry.

Q172 QUESTION

How do you configure provider authentication?

ANSWER

Providers support multiple auth methods. AWS: env vars (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`), shared credentials file, or IAM instance roles. Azure: service principal or managed identity. GCP: service account key or application default credentials. Never hardcode credentials in `.tf` files.

Q173 QUESTION

What is a provider alias?

ANSWER

Aliases allow multiple configurations of the same provider: `provider "aws" { alias="east"; region="us-east-1" }` and `provider "aws" { alias="west"; region="us-west-2" }`. Used for multi-region, multi-account, or any scenario needing different provider settings simultaneously.

Q174 QUESTION

What is provider inheritance in modules?

ANSWER

Modules inherit the default provider from the root module automatically. For aliased providers, pass explicitly: `module "compute" { providers = { aws = aws.east } }`. Best practice: modules should NOT declare their own provider configs – receive via `providers` argument from caller.

Q175 QUESTION

What is the `required_providers` block?

ANSWER

Declares required providers and version constraints: `terraform { required_providers { aws = { source="hashicorp/aws"; version="~>5.0" } } }`. Always declare in every module even if inherited. Enables explicit dependency management and reproducible builds via the lock file.

Q176 QUESTION

What is the terraform providers schema command?

ANSWER

terraform providers schema -json outputs the complete schema for all configured providers — all resources, attributes, types, and descriptions. Used by IDE plugins, documentation generators, and any tooling needing programmatic access to provider capabilities.

Q177 QUESTION

How do you use a local filesystem provider?

ANSWER

For development/testing of custom providers, reference from local path in required_providers: source = "example.com/myorg/myprovider". Configure filesystem mirror in ~/.terraformrc: provider_installation { filesystem_mirror { path = "/path/to/providers" } }.

Q178 QUESTION

What are data-only provider configurations?

ANSWER

Some providers require configuration even when only using data sources (e.g., specifying region). Provider config is always required if any resources or data sources from that provider are used. The provider block can be minimal: provider "aws" { region = var.region }.

Q179 QUESTION

What is the difference between provider version and plugin version?

ANSWER

They are the same thing. Each Terraform provider is a separate binary plugin with its own version. Provider versions are independent of the Terraform CLI version. The .terraform.lock.hcl records provider binary checksums to ensure integrity alongside version pinning.

16 Scenario-Based Questions

21 Qs

Q180 QUESTION

How do you handle a corrupted state file?

ANSWER

Steps: (1) DO NOT run plan or apply. (2) Restore from backup (versioned S3 bucket). (3) If no backup: use 'terraform state list' to see what's visible. (4) Use 'terraform import' to reimport critical resources. (5) For minor corruption: 'terraform state push' a manually repaired state file.

Q181 QUESTION

How do you refactor a resource into a module without downtime?

ANSWER

Use moved blocks (v1.1+): `moved { from = aws_instance.web; to = module.compute.aws_instance.web }`. Run 'terraform plan' — should show 0 changes. Run 'terraform apply' — updates state only. Remove moved block in a follow-up commit. Infrastructure never touched.

Q182 QUESTION

How do you manage multiple AWS accounts with Terraform?

ANSWER

Use provider aliases with `assume_role`: `provider "aws" { alias="prod"; assume_role { role_arn = "arn:aws:iam::123456789:role/Terraform" } }`. Reference in resources: `provider = aws.prod`. Use separate workspaces or configurations per account for isolation and access control.

Q183 QUESTION

How do you handle provider upgrades safely?

ANSWER

(1) Update version constraint in `required_providers`, (2) Run 'terraform init -upgrade', (3) Run 'terraform plan', (4) Review all changes — upgrades can add defaults that change existing resources, (5) Apply in non-production first, (6) Commit updated `.terraform.lock.hcl`.

Q184 QUESTION

How do you share data between two separate Terraform configurations?

ANSWER

Use `terraform_remote_state` data source to read outputs from another config's state file. The producing config must declare outputs. The consuming config reads them: `data.terraform_remote_state.vpc.outputs.vpc_id`. Both configs must have access to the same backend.

Q185 QUESTION

How do you prevent a team member from accidentally destroying production?

ANSWER

(1) `lifecycle { prevent_destroy = true }` on critical resources, (2) Use Terraform Cloud with RBAC — restrict who can approve runs, (3) Sentinel policies to block destroys, (4) Require PR approval for Terraform changes, (5) Use `-target` cautiously, (6) Separate production state/backend with restricted access.

Q186 QUESTION

How do you handle a stuck/failed apply?

ANSWER

(1) Check if state is locked — use 'terraform force-unlock ' only if no operation is running. (2) Check which resources were partially created in state. (3) Fix the root cause (IAM permissions, API limits, config errors). (4) Re-run 'terraform apply' — Terraform will pick up from where it failed.

Q187 QUESTION

How do you create resources in multiple regions?

ANSWER

Use provider aliases: provider "aws" { alias="east"; region="us-east-1" } provider "aws" { alias="west"; region="us-west-2" }. Then reference: resource "aws_s3_bucket" "east" { provider=aws.east } resource "aws_s3_bucket" "west" { provider=aws.west }. Or use modules that accept provider configuration.

Q188 QUESTION

How do you test Terraform modules?

ANSWER

(1) Use 'terraform validate' for syntax, (2) 'terraform plan' against a test environment, (3) Write .tftest.hcl files with 'terraform test' (v1.6+), (4) Use Terratest (Go testing framework) for integration tests, (5) Use checkov or tfsec for static security analysis.

Q189 QUESTION

How do you handle large Terraform configurations with many resources?

ANSWER

Split into multiple stacks by layer: networking, security, compute, data. Use modules for repeated patterns. Use remote state references for cross-stack data. Use Terraform Cloud workspaces per stack. This reduces blast radius, improves plan speed, and enables team ownership per layer.

Q190 QUESTION

How do you implement zero-downtime deployments with Terraform?

ANSWER

Use create_before_destroy = true in lifecycle block for resources that need replacing. For load-balanced applications: (1) Create new instances, (2) Register with load balancer, (3) Wait for health checks, (4) Deregister and destroy old instances. Use blue-green deployments for database schema changes.

Q191 QUESTION

How do you handle Terraform in a CI/CD pipeline?

ANSWER

Best practice pipeline: (1) terraform fmt -check (fail on formatting), (2) terraform init, (3) terraform validate, (4) terraform plan -out=tfplan (save plan), (5) Manual approval gate for production, (6) terraform apply tfplan (apply saved plan). Use Terraform Cloud for built-in pipeline support.

Q192 QUESTION

How do you manage drift in production?

ANSWER

Schedule regular 'terraform plan -refresh-only' runs in CI to detect drift. Alert when drift is found. Options: (1) Correct drift by re-applying Terraform (planned change), (2) Accept drift using ignore_changes, (3) Update Terraform config to match reality. Never silently ignore production drift.

Q193 QUESTION

How do you handle large teams with Terraform?

ANSWER

Strategies: (1) Module library with private registry for approved patterns, (2) Separate workspaces per team/environment, (3) Sentinel policies for governance, (4) Pull request reviews for all Terraform changes, (5) Terraform Cloud RBAC for access control, (6) Split into domain-specific stacks owned by respective teams.

Q194 QUESTION

How do you roll back a failed Terraform apply?

ANSWER

Terraform doesn't have built-in rollback. Options: (1) Restore previous state from versioned backend (S3 versioning or TF Cloud run history), then re-apply old config. (2) Manually fix the issue and re-apply. (3) For critical resources: restore from infrastructure backup. Prevention: use saved plans and approval gates.

Q195 QUESTION

How do you test Terraform configuration before production?

ANSWER

Testing strategy: (1) terraform validate — syntax, (2) tfsec / checkov — static security analysis, (3) terraform plan in staging — compare to expected, (4) terratest (Go) — integration tests against real infrastructure, (5) terraform test (v1.6+) — built-in test framework, (6) Cost estimation in TF Cloud.

Q196 QUESTION

What is the best way to manage Terraform versions across teams?

ANSWER

Use tfenv (Terraform version manager) or asdf to switch CLI versions. Enforce version in config: required_version = "~> 1.6". Use a .terraform-version file in project root (tfenv reads this automatically). Pin exact versions in CI/CD. Upgrade versions deliberately after testing in non-production.

Q197 QUESTION

What is Terraform's execution order for resource operations?

ANSWER

Terraform processes resources in dependency order based on the DAG. Resources with no dependencies can run in parallel (up to -parallelism limit). A resource waits only for its direct dependencies to complete. Destroy order is the reverse of create order. Use terraform graph to visualize execution order.

Q198 QUESTION

What is the difference between 'terraform destroy' and removing config?

ANSWER

terraform destroy explicitly destroys all managed resources. Removing a resource block from config and running terraform apply achieves the same result — Terraform notices the resource is in state but not in config and destroys it. Both approaches are valid; terraform destroy is more explicit for complete teardown.

Q199 QUESTION

What is a data source limitation compared to a resource?

ANSWER

Data sources are read-only and cannot modify infrastructure. Data source values are fetched at plan time (not apply time) unless they depend on resources being created. Data sources from external APIs may return stale data if the external system changes between plan and apply.

Q200 QUESTION

How do you handle API rate limits in Terraform?

ANSWER

Reduce -parallelism flag value (default 10) to limit concurrent API calls. Use retry logic in provider config where supported (e.g., AWS provider has retry_mode and max_attempts). Add depends_on to serialize operations. Use exponential backoff. Contact cloud provider to increase limits if needed.